

Effectiveness of a Generate–Verify–Repair Pipeline with Batfish for LLM-Generated Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We report a preregistered paired experiment (cnsms2026-001-gvr-batfish-prereg-v1) that evaluated whether a generate–verify–repair (GVR) pipeline—shared initial LLM candidate (gpt-5-mini) + a deterministic Batfish-style validator + at most one automated LLM repair—reduces deterministic-verifier detected semantic/safety failures on intent→vendor-CLI tasks. Planned N=300 pairs; 266 complete pairs were analysed after preregistered exclusions. Baseline pass-rate = 260/266 (97.744%); guarded (final GVR) pass-rate = 265/266 (99.624%). Paired difference = 0.01879699248; exact McNemar two-sided $p = 0.0625$. Paired bootstrap (B=10,000, seed=1729) 95% percentile CI = [0.0037593984962406013, 0.03759398496240601]. Repair was invoked for 6 tasks; median repair iterations per task = 0. Per-episode latency was not recorded in per-task artifacts and thus could not be evaluated. Under shared_initial_candidate semantics the observed absolute improvement is small and did not meet the preregistered $\alpha=0.05$ decision rule. We provide preregistered design details, deterministic validator semantics, exact paired counts, representative artifact-grounded repair examples, contamination findings (no exact public duplicates flagged), and a focused discussion of limitations and implications.

I. INTRODUCTION

Generative large language models (LLMs) are actively explored for NetOps tasks such as intent→CLI translation, zero-touch configuration synthesis, and automated maintenance workflows [1], and surveys emphasize both opportunity and risk when LLMs are applied to operational network configuration [2]. Stochastic LLM outputs can produce syntactic or semantic violations—missing required commands, unsafe command orderings, or constraint breaches—that create operational risk if applied directly. To mitigate that risk, pipelines that interpose deterministic verifiers (e.g., Batfish or header-space analyses) and bounded repair loops—collectively generate–verify–repair (GVR)—have been proposed to provide deterministic acceptance criteria and automated correction [3,4]. Prior work presents components and prototypes, but preregistered, paired evaluations that archive verifier traces and quantify verifier-triggered repair benefit on reproducible NetOps task banks remain limited.

This study tested the preregistered question (cnsms2026-001-gvr-batfish-prereg-v1): Does a GVR pipeline that uses a single sampled initial LLM candidate per task (shared_initial_candidate semantics), a deterministic Batfish-style validator, and at most one automated LLM repair call reduce deterministic-verifier detected semantic/safety failures relative to the same initial candidate without repair?

Our principal contribution is a preregistered, artifact-archived paired evaluation executed under the CNSM Agentic AI Researcher constraints that reports exact paired counts, inferential outcomes, execution accounting, representative artifact examples, and an evidence-grounded interpretation.

II. RELATED WORK

LLM applications in NetOps include intent→CLI translation systems and focused benchmarks studying automated configuration synthesis and correctness [1]. Surveys synthesize common failure modes (including hallucination and constraint violations) and advocate grounding and verification components for automation [2]. Generate–verify–repair architectures have been proposed and explored in code generation and regulated domains to achieve deterministic validation via external oracles and bounded repair loops [3]. Network verification tools such as Batfish and header-space analyses provide deterministic reachability and ordering checks and are natural verifier choices in LLM-augmented NetOps prototypes; prior analyses document Batfish’s design and motivate its integration in automated pipelines [4]. Work examining what LLMs need to synthesize correct router configurations highlights sequencing and vendor-syntax difficulties that verifiers can detect and flag for repair [5].

Our study differs by executing a preregistered paired test that (a) uses shared_initial_candidate semantics exactly as registered, (b) archives raw model and validator traces for auditability, and (c) reports exact paired contingency counts together with the preregistered exact McNemar test and a paired bootstrap CI to aid interpretation in a small-discordant sample.

III. METHODOLOGY

Overview and estimand. We preregistered the study as cnsms2026-001-gvr-batfish-prereg-v1 and analysed the paired estimand labeled paired_success_rate_difference_guarded_minus_baseline under shared_initial_candidate semantics. Under those semantics the same single sampled initial model candidate (per task) is used to compute both outcomes: baseline = validator pass/fail on the initial candidate; guarded = final validator pass/fail after the allowed validator-triggered repair (at most one LLM repair call). This design isolates the marginal effect of the permitted validator-triggered repair

for a fixed initial candidate rather than conflating effects of different initial samples or different first-pass prompting strategies.

Task generation, inclusion/exclusion, and determinism. The preregistered task bank deterministically produced 300 intent→CLI pairs composed of public NetConfEval-style examples and synthetic tasks generated by a seeded deterministic generator. Tasks are stratified into two vendor-dialect clusters (150 Cisco-like, 150 Juniper-like). Generator IDs, seeds, and per-task payloads (initial_state, expected_final_state, required_sequence, allowed_touched_fields, and workflow_policies) are archived in the task manifest. Inclusion/exclusion rules and the retry policy followed preregistration: transient provider/API failures were retried up to two times; pairs were excluded from the primary confirmatory analysis only when both arms failed due to infrastructure/provider errors consistent with the preregistered missingness_plan. All task selection indices were fixed before model outcomes were observed and are archived to ensure deterministic replay of the task bank.

Model, orchestration and recorded runtime parameters. The declared model used in the run was gpt-5-mini (execution/model_configuration.json). Archived execution metadata records max_output_tokens = 1024, maximum_attempts_per_call = 1, provider = openai_responses, and temperature = null (unset). Provider accounting in the deterministic reconciliation artifact records hosted_model_call_event_count = 306, completed_hosted_model_call_event_count = 272, and failed_hosted_model_call_event_count = 34; stage accounting shows shared_initial_generation = 300 and repair = 6. Because the provider configuration recorded temperature as unset and no centralized deterministic sampling seed is available for hosted calls, nondeterministic sampling of provider outputs cannot be excluded; we disclose this operational nondeterminism and treat it as a limitation in the Results and Disclosure.

Deterministic validator semantics and scoring rules. The binary oracle was deterministic_netops_validator_v5 and it applied a fixed battery of checks recorded in per-episode validator traces. For each candidate the validator (1) parsed and normalized the CLI commands (producing parsed_command_count and normalized_configuration), (2) checked the required_sequence field against parsed commands (required_command_count), (3) enforced constrained_touch policies (allowed_fields and allowed_touched_fields), and (4) executed reachability/constraint checks on the synthetic topology model to detect sequencing or dependency violations. The validator output includes observed_touched_field_count, violation_codes, and final valid boolean; the primary endpoint was the validator's binary pass/fail under the scoring rule full_intent_constraint_validation. Every episode archives both validation_before and validation_after traces so individual failure modes are inspectable and replayable.

Repair loop mechanics (bounded and archived). The preregistered repair stage is strictly bounded to at most one

automated LLM repair call per task when the initial candidate fails the validator. The repair prompt construction is deterministic given the validator failure codes and the task payload: it includes the task constraints, the normalized initial candidate, and explicit violation codes, and it requests a corrected configuration that satisfies the validator battery. Repair provider traces and repair_prompt_sha256 values are archived when invoked; stage accounting reports six repair calls in the run. Repair outputs are submitted to the same deterministic validator and the guarded final outcome is the validator result after that single repair attempt. Limiting repairs to one invocation preserves a clear estimand and enforces a bounded repair budget as preregistered.

Analysis procedures and exact inferential specification. The prespecified primary inferential test is the exact McNemar two-sided test on paired binary outcomes at $\alpha=0.05$. To quantify effect magnitude and uncertainty we also preregistered a paired bootstrap percentile confidence interval with $B = 10,000$ resamples and seed = 1729; those exact bootstrap parameters are recorded in analysis/analysis_log.jsonl. Missingness handling followed the preregistered complete_pair_primary rule: pairs where both arms failed due to provider infrastructure issues were excluded from the primary test (documented in analysis/missingness_summary.csv), and conservative sensitivity analyses treating excluded pairs as failures were preserved as archived artifacts.

Accounting, reconciliation and reproducibility artifacts. Execution accounting recorded planned_episode_count = 600 and completed_episode_count = 532, with failed_episode_count = 68; after applying preregistered exclusions the analysed complete_pair_count = 266. Deterministic reconciliation artifacts verify internal consistency of paired counts and provider calls (provider and stage counts, cache behavior, and cross-condition accounting) and assert that deterministic consistency checks passed. Key artifact categories archived for reproducibility include raw model responses, provider call traces, per-episode scoring JSONs (validator traces), the task manifest, transformation manifests, model_configuration.json, execution/raw_results.jsonl, and the analysis result artifacts (condition_summary.csv, paired_contingency_table.csv, missingness_summary.csv, contamination_summary.csv, and the paired_difference_figure.svg). File paths and manifests are provided with the execution bundle to enable exact reanalysis and targeted replay of repaired episodes.

Operational measurements attempted and absent. The preregistration included secondary estimands for median_repair_iterations_per_task and median end-to-end latency; repair iterations were measurable and recorded (repair calls = 6; median repair iterations per task = 0). Per-episode end-to-end latency (generation→validation→repair→final validation) was not recorded in the archived per-task artifacts and therefore the preregistered latency estimand could not be evaluated; this absence is noted as a preregistered deviation in the Limitations and drives a concrete reproducibility recommendation to include timing instrumentation in future runs.

Threats to validity embedded in the design. The methodology preserves explicit distinctions required by the preregistration: internal validity is governed by shared_initial_candidate semantics (the estimand measures repair benefit for a fixed sample rather than differences from alternative first-pass strategies), construct validity depends on validator rule coverage (the deterministic_netops_validator_v5 battery defines which semantic violations are detectable), and external validity is limited by the two vendor clusters and synthetic topology scale. The preregistration’s missingness rules and sensitivity analyses were applied exactly and all exclusions, retries, and provider failures are archived in the execution manifest to support transparent effect-size interpretation.

IV. RESULTS

Execution accounting and sample construction. The preregistered run planned 600 model-condition episodes (300 tasks $\times$ 2 conditions). The orchestrated pipeline completed 532 episodes and recorded 68 failed episodes (execution_manifest.json). After applying preregistered exclusion rules that remove pairs affected by both-arm provider/infrastructure failures, the confirmatory analysis used 266 complete paired units (analysis/missingness_summary.csv). Key provider-call accounting from the deterministic reconciliation artifact: hosted_model_call_event_count = 306 total model call events, of which 272 completed successfully and 34 failed; cache was unused (cache_hit_count = 0, cache_miss_count = 272). Stage accounting shows that a shared initial generation was produced for every task (shared_initial_generation = 300) and the automated repair stage executed in 6 episodes (repair = 6). All execution-level manifests and per-episode traces are archived to permit replay and audit.

Primary paired outcomes and exact counts. The confirmatory paired contingency (guarded vs baseline) computed on the 266 complete pairs is shown in analysis/paired_contingency_table.csv and summarized here: n_{11} = 260 (both baseline and guarded pass), n_{10} = 5 (guarded pass, baseline fail), n_{01} = 0 (baseline pass, guarded fail), n_{00} = 1 (both fail). These margins yield observed success rates baseline = $260/266 = 0.9774436090$ and guarded = $265/266 = 0.9962406015$ with an absolute paired difference of 0.01879699248.

Inferential results: exact McNemar and paired bootstrap CI. Per the preregistered primary test, the exact McNemar two-sided p-value for the observed discordant counts ($n_{10}=5$, $n_{01}=0$) is $p = 0.0625$, which does not meet the preregistered $\alpha = 0.05$ decision rule for rejection. To quantify effect magnitude uncertainty we computed the preregistered paired bootstrap percentile 95% CI with $B = 10,000$ resamples and seed = 1729 (analysis/analysis_log.jsonl); the resulting CI is [0.0037593984962406013, 0.03759398496240601], narrowly excluding zero. Reporting both results preserves the preregistered decision rule while also conveying interval evidence: the discrete nature of McNemar’s exact two-sided test with few discordants can produce conservative p-values even when

bootstrap resampling indicates a small positive paired difference with limited sampling variability. Both inferential artifacts (exact p and bootstrap CI with documented seed and resamples) are archived.

Repair invocation behavior and representative repaired episodes. The repair stage executed for 6 of the 300 tasks (stage_counts: repair = 6). Median repair iterations per task across the analysed sample is 0 (majority of tasks required no repair because the shared initial candidate already passed validation). Representative repaired episode: task-000008. The shared initial candidate for task-000008 (execution/responses/task-000008-shared-initial.txt) contained a truncated final token and failed validator checks (validation_before showed parsed_command_count < required_command_count). The automated repair call produced a corrected candidate (execution/responses/task-000008-guarded.txt) whose validator trace (execution/scoring/task-000008-guarded.json) records validation_after.valid = true with no violation codes. Per-episode scoring JSONs record whether a repair was applied, the repair_prompt reference, and the final normalized_configuration accepted by the deterministic validator; all such per-episode repair traces are archived for examination.

Contamination and sensitivity to missingness. The preregistered contamination detector flagged no exact public duplicates (analysis/contamination_summary.csv empty), so no tasks were excluded from the primary analysis on contamination grounds. The preregistered missingness policy excluded 34 both-arm failed episodes from the primary test; a sensitivity analysis that conservatively treats those 34 excluded pairs as failures (i.e., complete $N = 300$ with both arms set to failure where missing) yields baseline = $260/300 = 0.866667$ and guarded = $265/300 = 0.883333$, but retains the same discordant structure ($n_{10}=5$, $n_{01}=0$) leading to the same exact McNemar $p = 0.0625$. Thus, under the preregistered conservative missingness treatment the formal decision does not change; archived missingness_summary.csv and deterministic_reconciliation.json provide the complete accounting.

Supplementary quantitative artifacts and reproducibility. Analysis artifacts included in the evidence bundle and referenced above provide the reproducible numeric foundations for these results: analysis/condition_summary.csv (baseline: 260 successes, 6 failures; guarded: 265 successes, 1 failure), analysis/paired_contingency_table.csv (n_{11} , n_{10} , n_{01} , n_{00} table), analysis/missingness_summary.csv (counts of excluded and failed episodes), and analysis/analysis_log.jsonl (bootstrap settings: $B=10,000$, seed=1729). deterministic_reconciliation.json documents provider call totals and stage counts that explain model_calls_used = 306 and repair_calls = 6. All files and their hash manifests are archived to enable exact reanalysis or targeted follow-up audits.

V. DISCUSSION

Statistical interpretation and small-discordant behavior: The guarded GVR pipeline produced a small absolute improve-

ment (~ 1.88 percentage points). Under the preregistered exact McNemar decision rule the improvement did not reach $\alpha=0.05$ ($p=0.0625$). The paired bootstrap CI that excludes zero highlights finite-sample discreteness: with few discordants ($n_{10}=5$, $n_{01}=0$) McNemar’s discrete two-sided p -value can be conservative for small effects while bootstrap percentile CIs reflect resampled paired-difference variability. We report both inferential perspectives so readers may weigh formal hypothesis rejection against effect magnitude and CI width.

Operational implications and boundary conditions grounded in artifacts: Baseline first-pass accuracy was high ($\approx 97.7\%$), leaving few failures for the repair stage to correct—hence rare repair invocations (6/300). Stage accounting and provider traces show repair-stage overhead was small in the observed run, but we did not archive per-episode latency and therefore cannot quantify repair latency or end-to-end timing against the preregistered operational bound (median ≤ 60 s). The low repair rate suggests bounded automated repair adds limited marginal benefit on high-first-pass corpora; however, GVR may yield larger absolute improvements when first-pass accuracy is lower, verifier coverage expands, or when the verifier detects failure modes common in production workloads.

Threats to validity (artifact-grounded): Internal validity: the `shared_initial_candidate` estimand measures the effect of validator-triggered repair for a fixed initial sample and does not assess independent multiple draws or constrained first-pass prompting because independent first-pass arms were not executed under the preregistration. Construct validity: `deterministic_netops_validator_v5` enforces a finite, archived rule battery (syntactic parsing, `required_sequence`, `constrained_field` checks, reachability) and therefore semantic violations outside that battery are undetected and unmeasured. External validity: tasks derive from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor dialects; results may not generalize to other vendors, larger topologies, or production heterogeneous workloads. Conclusion validity: the loss of 34 both-arm provider failures reduced effective N and the observed few discordant pairs limit power for small effect detection.

Design lessons and recommendations grounded in execution artifacts: (1) When baseline pass rates are high, design experiments targeting lower-baseline corpora or include arms that modify first-pass sampling (e.g., multiple draws or constrained prompts) to increase discordant counts and power. (2) Archive per-episode pipeline timing (request $\rightarrow$ generation $\rightarrow$ validation $\rightarrow$ repair $\rightarrow$ final validation) to assess operational latency bounds; our run lacked per-episode latency and thus the preregistered latency estimand could not be evaluated. (3) Expand and publish validator rule coverage and include tests that exercise verifier false negatives; per-episode validator traces permit such audits and should be part of future preregistrations. (4) Consider paired designs that explicitly compare `shared_initial_candidate` semantics to independent generation semantics in separate preregistered arms; such contrasts require independent initial draws and were outside this run by design.

Reproducibility and archival resources: All raw responses, execution/provider call traces, validator traces, scoring JSONs, analysis artifacts (paired contingency table, condition summary, missingness summary, contamination summary), `deterministic_reconciliation.json`, and `model_configuration.json` are archived and hashed in the evidence bundle. The archived `analysis_log.jsonl` records bootstrap settings ($B=10,000$, $\text{seed}=1729$). These artifacts permit exact reexecution of the confirmatory analysis and targeted replay of repaired episodes.

VI. LIMITATIONS

- `Shared_initial_candidate` semantics: both arms used the same single sampled initial candidate per preregistration; the estimand measures validator-triggered repair benefit for that fixed initial sample and does not evaluate independent first-pass generations or constrained first-pass prompting strategies.
- Missingness and sample reduction: planned $N=300$ pairs; after infrastructure/provider exclusions 266 pairs were complete and 34 both-arm episodes were excluded per preregistered rules, reducing effective sample size and precision.
- Small discordant counts: primary inference used exact McNemar with few discordants ($n_{10}=5$, $n_{01}=0$); conclusions are sensitive to inferential choice and limited power.
- Validator coverage: `deterministic_netops_validator_v5` enforces a finite set of syntactic/semantic/reachability rules; semantic violations outside its coverage are possible and unmeasured.
- Runtime nondeterminism and sampling: archived `execution/model_configuration.json` records `temperature = null` and no deterministic seed; nondeterministic sampling cannot be excluded for recorded model calls.
- H2 latency not evaluated: per-episode end-to-end latency was not present in archived per-task artifacts and therefore the preregistered latency bound (median ≤ 60 s) could not be assessed.
- External validity: task corpus derives from public NetConfEval-style examples and a seeded synthetic generator limited to two vendor-dialect clusters; results do not imply universal benefit across other vendors, larger topologies, or production deployments.

VII. CONCLUSION

We executed a preregistered paired experiment that evaluated a GVR pipeline (`gpt-5-mini` + `deterministic_netops_validator_v5` + ≤ 1 automated repair) on intent $\rightarrow$ CLI tasks under `shared_initial_candidate` semantics. Across 266 analysed pairs the guarded pipeline improved validator pass rate by 0.01879699248 but did not meet the preregistered exact-McNemar $\alpha=0.05$ decision rule ($p=0.0625$); a paired bootstrap CI ($B=10,000$, $\text{seed}=1729$) narrowly excludes zero. Repair calls were rare (6/300) and median repair iterations per task = 0. Per-episode latency was not archived and could not be assessed. All execution and analysis artifacts (raw responses, validator traces, provider

call accounting, and reconciliation manifests) are archived in the evidence bundle to support reanalysis and follow-on work.

References [1] C. Wang, M. Scazzariello, A. Farshin, S. Ferlin, D. Kostić, and M. Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," in Proc. ACM, 2024, doi: 10.1145/3656296. [2] S. Y. Long, J. Tan, B. Mao, F. Tang, Y. Li, M. Zhao, and N. Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models," IEEE Commun. Surveys & Tutorials, 2025, doi: 10.1109/comst.2025.3526606. [3] R. Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments," SSRN, 2026, doi: 10.2139/ssrn.6754899. [4] M. Brown, A. Fogel, D. Halperin, V. Heorhiadi, R. Mahajan, and T. Millstein, "Lessons from the evolution of the Batfish configuration analysis tool," Proc., 2023, doi: 10.1145/3603269.3604866. [5] R. Mondal, A. Tang, R. Beckett, T. Millstein, and G. Varghese, "What do LLMs need to Synthesize Correct Router Configurations?," Proc., 2023, doi: 10.1145/3626111.3628194.

DISCLOSURE STATEMENT

Disclosure (concise): Declared model: gpt-5-mini (execution/model_configuration.json archived; declared_model_version = "gpt-5-mini"). Orchestration adapter: hosted_netops_gvr_v1. Deterministic validator: deterministic_netops_validator_v5. Preregistration: cnsm2026-001-gvr-batfish-prereg-v1 (manifest SHA-256: 46a223492f760950b3d06f475129ce21e52e296c4c2a1e09df3ccd50bec9f891). Initial master prompt immutable reference: provenance/master_prompt.txt (SHA-256: 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Human role and autonomy boundary: before the autonomy lock a human Research Director selected the topic family and approved pre-lock infrastructure development; after the autonomy lock the registered pipeline autonomously performed literature selection, preregistration, experimental execution, analysis, manuscript drafting, AI peer review simulation, and revision. Nondeterminism disclosure: archived execution/model_configuration.json records temperature = null (unset) and no deterministic sampling seed is archived; therefore nondeterministic sampling of model outputs cannot be excluded for the recorded provider calls. Execution and analysis artifacts, logs, and hash manifests are archived in the evidence bundle referenced in the preregistration.

REFERENCES

- [1] Changjie Wang, Mariano Scazzariello, Alireza Farshin, Simone Ferlin, Dejan Kostić, Marco Chiesa, "NetConfEval: Can LLMs Facilitate Network Configuration?," 2024, 10.1145/3656296
- [2] S.Y. Long, Jingjing Tan, Bomin Mao, Fengxiao Tang, Yangfan Li, Ming Zhao, Nei Kato, "A Survey on Intelligent Network Operations and Performance Optimization Based on Large Language Models", 2025, 10.1109/comst.2025.3526606

- [3] Rafael Cadenas, "Convergent Correctness in Stochastic Code Generation: A Generate-Verify-Repair Architecture for Deterministic Validation of Autonomous AI Coding Agents in Regulated Environments", 2026, 10.2139/ssrn.6754899
- [4] Matt Brown, Ari Fogel, Daniel Halperin, Victor Heorhiadi, Ratul Mahajan, Todd Millstein, "Lessons from the evolution of the Batfish configuration analysis tool", 2023, 10.1145/3603269.3604866
- [5] Rajdeep Mondal, Alan Tang, Ryan Beckett, Todd Millstein, George Varghese, "What do LLMs need to Synthesize Correct Router Configurations?", 2023, 10.1145/3626111.3628194